BigTech Electronics Store

Project Report

Prepared by: Harika Reddy

Abstract

This report presents an Indian electronics shopping project with 28 synthetic products across seven categories, website scoped Ezzie support, persistent browser inventory and a Java Spring Boot order service. The customer journey covers discovery, cart, checkout, stock conflict, payment failure, confirmation, an eight stage delivery timeline, order history and account availability.

The order design uses five deterministic agents coordinated through typed results. Successful payment commits reserved stock, payment failure releases it and a stock conflict rejects the reservation before payment. Ten JavaScript tests and four Java tests verify the deployed browser rules and canonical backend behavior.

Keywords: electronics retail; inventory reservation; Java; multi agent orchestration; delivery tracking; JavaScript; testing

Report structure

The report presents the problem, project scope, customer journey, system design, inventory and delivery controls, evaluation evidence, limitations, reproducibility details, and conclusion.

1. Problem statement, root cause and purpose

BigTech is an Indian electronics shopping application that connects product discovery, current stock, cart rules, simulated checkout, payment outcomes, inventory reservation, fulfilment, delivery tracking, order history, account information and Ezzie support. I selected this problem because a visible storefront alone does not demonstrate the decisions that protect a customer order.

The root cause in many compact demonstrations is disconnected state. A product card may show one available quantity while the cart uses the original quantity, a failed payment may reduce stock permanently, and a confirmed order may stop at an order number without showing how it reaches delivery.

The purpose of BigTech is to make the complete lifecycle reproducible within a focused personal project. The customer interface remains simple, while JavaScript and Java contracts preserve the inventory, payment, fulfilment and delivery evidence behind each result.

Objective	Implemented evidence
Connect catalogue to checkout	One stock map supports product cards, cart limits and final validation
Protect inventory	Reservation is committed, released or rejected by outcome

Objective	Implemented evidence
Explain delivery	Eight customer milestones with completed, current, upcoming and stopped states
Verify behavior	10 JavaScript tests, 4 Java tests and browser journey evidence

2. Scope and demonstration data

The catalogue contains 28 synthetic products across phones, laptops, televisions, audio, appliances, wearables and gaming. Records include a stable product identifier, category, price, list price, rating, badge and starting stock. Prices are displayed in Indian rupees and the delivery calculation uses an Indian six digit pincode.

The dataset is deliberately compact because BigTech tests software state transitions rather than training a statistical model. Increasing the catalogue to thousands of repeated products would not improve the evidence for reservation release, payment stopping rules or delivery milestones.

Five public order examples make out for delivery, shipped, delivered, payment failed and stock changed behavior visible on a first visit. Personal cart, order and inventory changes are versioned and stored only in the visitor's browser.

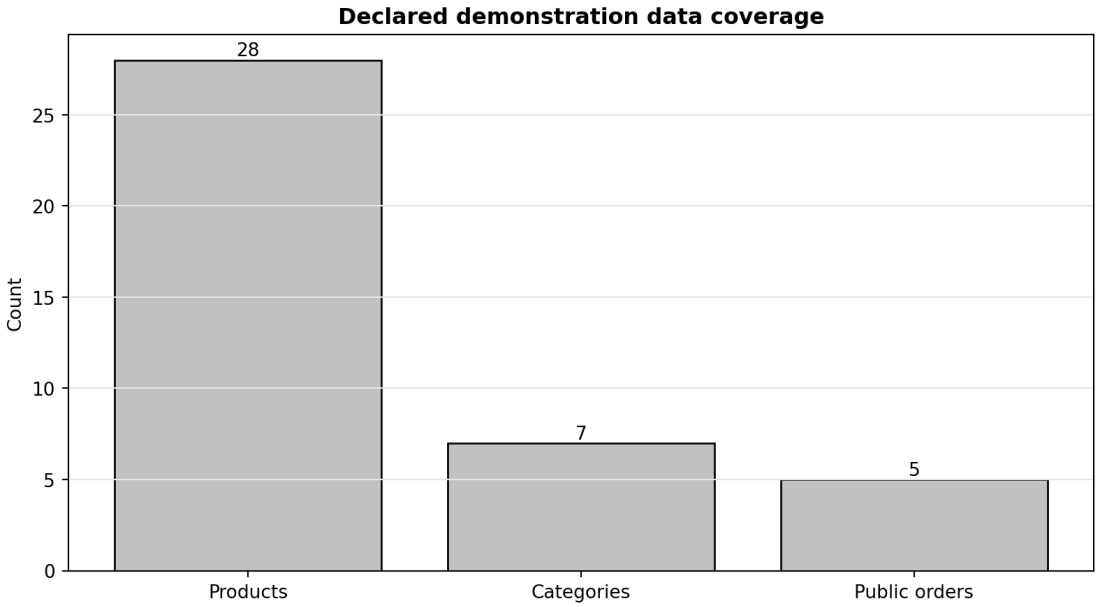

Figure 1. Declared catalogue, category and public order coverage.

Test question: Does the demonstration contain enough varied records to exercise discovery and order behavior without claiming analytical scale?

Observed result: Twenty eight products cover seven categories and five first visit order states.

Interpretation: The bounded dataset supports repeatable software tests and keeps every scenario easy to inspect.

3. Customer journey

A visitor can search, filter and sort the catalogue, review discount and stock, add an available item, change quantity within the current limit and inspect order totals. Ezzie can recommend an in stock product within a declared budget and add the suggestion to the same cart.

Checkout collects demonstration delivery details and provides three reproducible outcomes: successful payment, failed payment and an item becoming unavailable. Every result receives a saved order reference, including failures, so the stopping point and prevented side effects remain visible.

Confirmed orders open a detailed journey with received, reserved, paid, picking, packed, shipped, out for delivery and delivered stages. The page also shows the inventory quantity before checkout, the amount held and the amount available after the branch completes.

4. System architecture

The public application is written with semantic HTML, responsive CSS and JavaScript modules. Product data is separate from calculations and order rules. Versioned local storage holds personal cart, order and stock state so the Vercel demonstration works without collecting accounts or payment information.

A Java 17 Spring Boot service provides the independently testable order contract. OrderController validates the request and delegates to OrderCoordinator. The coordinator sends typed results through five bounded agents and returns inventory lines, customer milestones and an internal workflow trace.

The static browser workflow is contract compatible with the Java decisions. This is an explicit deployment boundary: the public site remains free and easy to inspect, while the backend can run locally or on a separate Java capable host.

Component	Responsibility	Evidence
Browser storefront	Customer interaction and persistent demo state	Search, cart, account and order screens
Order Coordinator	Required sequence and stopping rules	Typed branch response
Inventory Agent	Atomic all line reservation	Before, held and after quantities
Payment Agent	Approve or decline simulation	Completed or failed workflow step
Fulfilment and Delivery Agents	Estimate, packing and milestones	Delivery date and eight stages
Ezzie	Website scoped support	Catalogue, cart and order responses

5. Multi agent orchestration experiment

The backend uses Inventory Agent, Payment Agent, Fulfilment Agent, Delivery Agent and Notification Agent. These are deterministic Java components, not language model agents. Each owns one business decision and returns a typed workflow step to OrderCoordinator.

The coordinator follows dependency order. Payment cannot run before stock is held. Fulfilment and delivery cannot run before payment is approved. Notification runs for every branch because a customer needs a visible outcome even when the order stops.

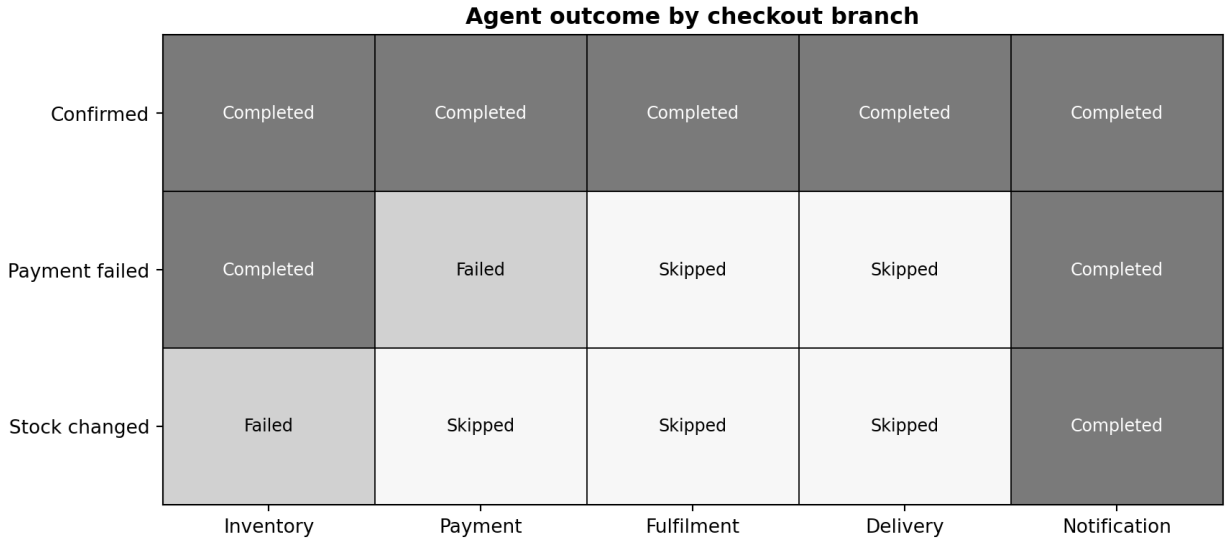

Figure 2. Completed, failed and skipped agent outcomes by checkout branch.

Test question: Does a required failure stop every downstream agent that could create an incorrect side effect?

Observed result: All five agents complete for confirmation. Payment failure skips fulfilment and delivery. Stock change skips payment, fulfilment and delivery while notification records the attempt.

Interpretation: The trace makes partial execution visible and proves that a generic error message is not hiding a continued workflow.

6. Inventory reservation experiment

InventoryService validates every cart line before modifying any quantity. This all line check prevents a partial reservation when one product is unavailable. The Java methods are synchronized so the reservation transition is atomic inside one service process.

A successful payment commits the held quantity. A failed payment calls release and restores the previous available value. An unavailable quantity returns a rejected disposition without changing stock or attempting payment.

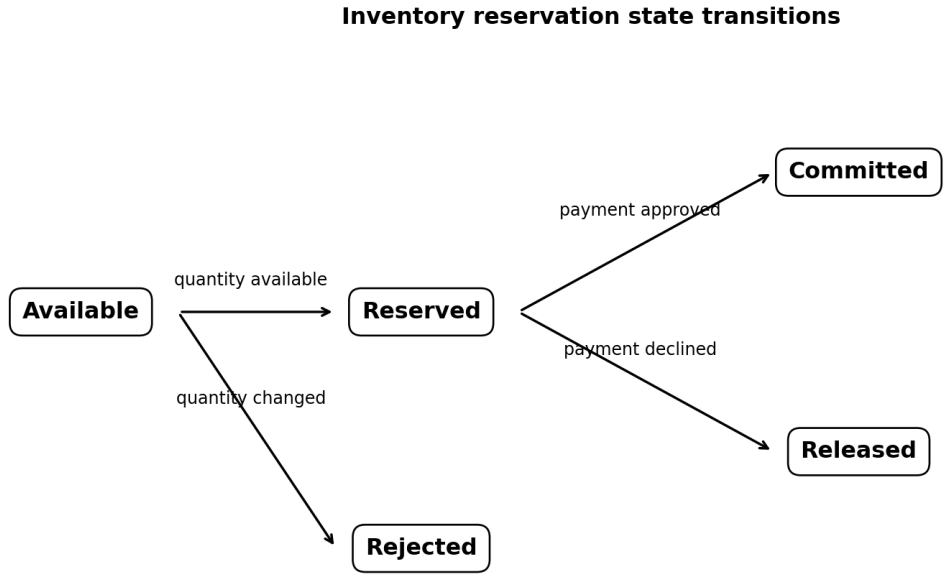

Figure 3. Available, reserved, committed, released and rejected inventory states.

Test question: Is inventory reduced only when payment succeeds and restored after a decline?

Observed result: Confirmation commits the reservation, failure releases it, and stock conflict rejects it before payment.

Interpretation: The response snapshot provides quantity evidence for each branch instead of relying on a success or failure label alone.

7. Delivery lifecycle experiment

Delivery is modeled as an ordered customer contract rather than one estimated date. The same fields support a newly confirmed order and advanced public examples. A current stage identifies active work, upcoming stages describe what follows and stopped stages explain why delivery was never created.

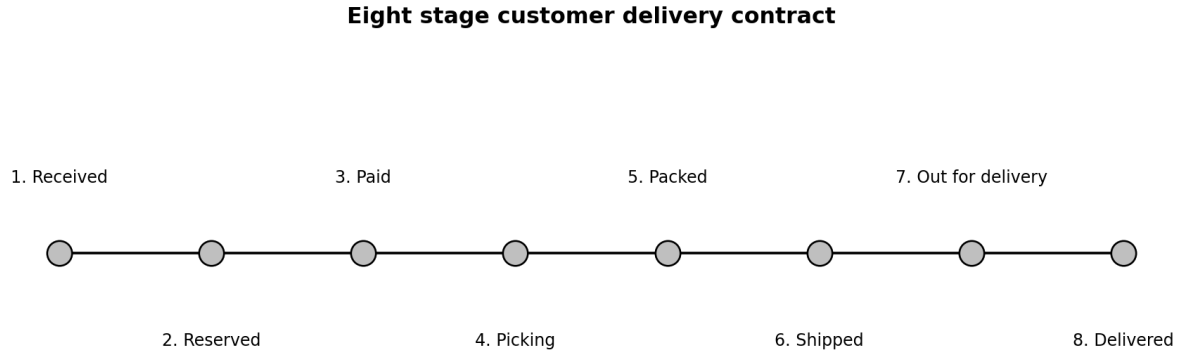

Figure 4. Eight ordered customer milestones from receipt through delivery.

Test question: Can the project represent the full post checkout journey in a consistent structure?

Observed result: Every confirmed response contains order received, items reserved, payment confirmed, picking, packed, shipped, out for delivery and delivered.

Interpretation: Public examples advance the same contract to shipped, out for delivery and delivered without introducing a second order model.

8. Scenario stopping experiment

The milestone position reached by each branch provides a customer centered view of failure. A confirmed order creates the complete contract and begins fulfilment. Payment failure reaches the payment stage but creates no shipment. Stock change stops at reservation and guarantees payment was not attempted.

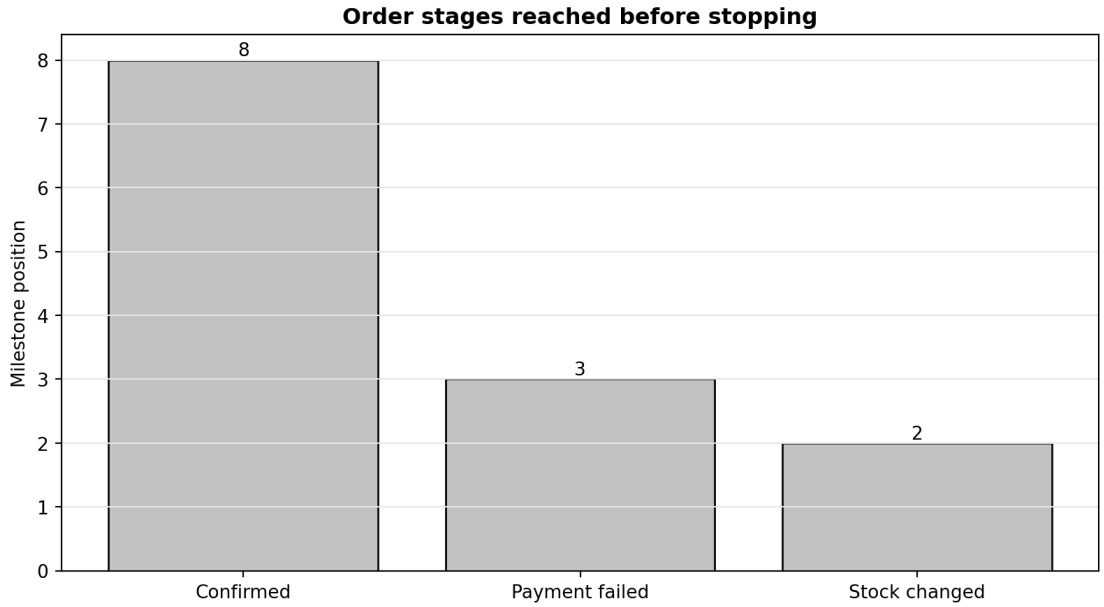

Figure 5. Relative milestone progress for confirmed, payment failed and stock changed branches.

Test question: Does the customer timeline stop at the same boundary as the internal workflow?

Observed result: Confirmation produces all eight milestones, payment failure stops at the third stage and stock change stops at the second.

Interpretation: Customer and technical evidence agree on where processing ended, reducing ambiguity in order support.

9. Automated verification experiment

The JavaScript suite verifies cart limits, delivery charges, payment recovery, inventory commitment, stock rejection, the eight stage timeline, catalogue coverage and Ezzie boundaries. The JUnit suite verifies Java commit, release, rejection and delivery milestone behavior using a fixed clock.

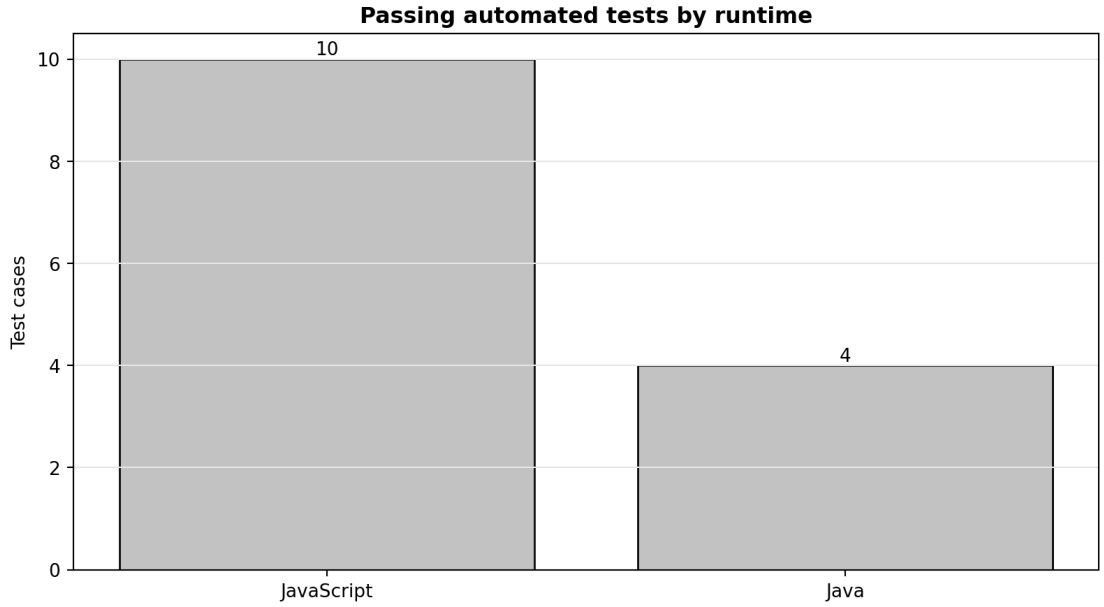

Figure 6. Passing automated tests across browser logic and Java service code.

Test question: Are both the deployed browser rules and canonical Java workflow checked independently?

Observed result: Ten JavaScript tests and four Java tests pass, giving 14 automated test cases.

Interpretation: The suite checks the state after failure, not only the displayed status. This includes preserved stock after decline and skipped payment after conflict.

10. Browser verification experiment

Browser verification starts with cleared storage and follows a product through cart, checkout, confirmation, order detail and persisted stock. It then reproduces failed payment and stock changed outcomes, opens the account availability summary and asks Ezzie for product and order support.

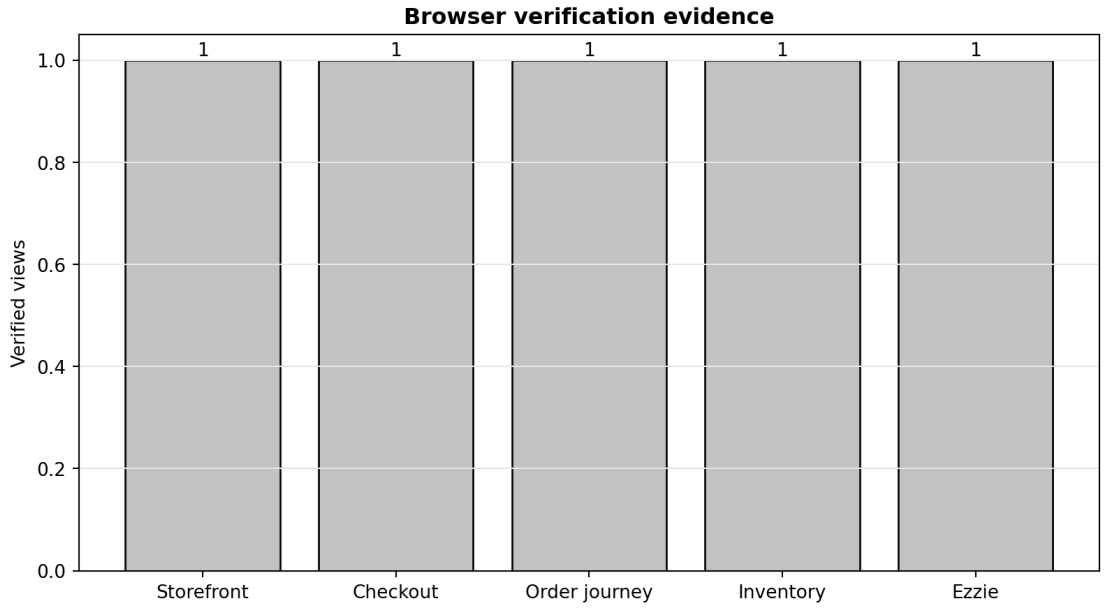

Figure 7. Customer views covered by browser verification and committed evidence.

Test question: Do the tested rules remain visible and understandable in the deployed customer interface?

Observed result: The verification covers storefront, checkout, order journey, inventory evidence and Ezzie support.

Interpretation: Browser evidence complements unit tests by checking navigation, rendered status and state persistence across screens.

11. Interface evidence

The storefront capture verifies the entry point used in browser testing. Category navigation, product artwork, prices, discount, stock, search, account and cart are visible without exposing internal workflow terminology. Separate captures in the evidence directory document checkout and Ezzie.

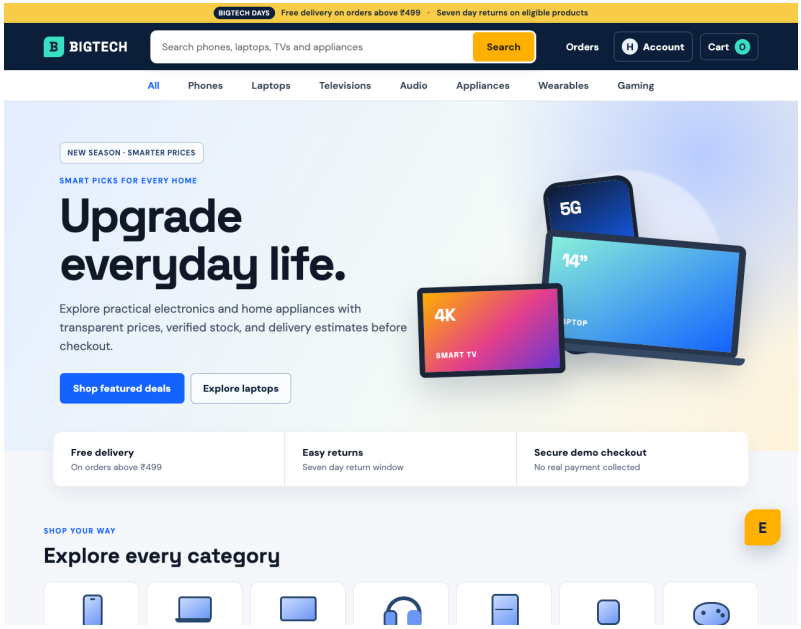

Figure 8. Deployed BigTech storefront used as the entry point to the verified order flow.

Test question: Is the electronics catalogue understandable and usable before the technical order rules are exercised?

Observed result: The interface presents seven categories, INR pricing, current stock, search, cart and account navigation in one responsive layout.

Interpretation: The customer experience remains a normal retail website while the repository provides deeper technical evidence.

12. Ezzie support boundary

Ezzie classifies requests for product discovery, budget, cart, inventory, delivery, returns, payment, order lookup and account help. Recommendations use the current stock map, so a product purchased to zero is not offered as an available choice. Order answers use the same saved milestone contract shown on the order detail page.

The assistant is deterministic and website scoped. It does not call a paid model, send customer messages to another service or invent general purpose answers. Outside requests receive a concise boundary response. This design is smaller than an open ended chatbot but more reproducible for a public software project.

13. Failure analysis

The strongest failure evidence is an absent side effect. A stock conflict must show that no unit was held and payment was skipped. A declined payment must show that held stock returned and no delivery date exists. Ezzie must not invent an order when an exact number is missing.

Every failed attempt is saved in order history with a specific reason and stopped timeline. This gives the customer an explanation and provides a durable object for browser verification and order lookup.

Failure	Safe result	Verification
Quantity unavailable	Reservation rejected and payment skipped	JavaScript and JUnit stock tests
Payment declined	Reservation released and delivery absent	Before and after equality assertion
Unknown order	No invented status	Ezzie exact lookup behavior
Outside request	Website scope response	Assistant boundary test

14. Limitations and responsible interpretation

Products, stock, customer information, payment results, orders and delivery events are synthetic. Browser local storage is not a shared database and must not be interpreted as warehouse inventory. The static public site does not call the local Spring Boot service.

The project does not contain authentication, database transactions, real payment, reservation expiry, courier integration, returns settlement, fraud checks or production monitoring. The Java synchronization only protects one service process and is not a replacement for database level concurrency control.

These limitations define a clear project boundary. The implemented evidence supports claims about deterministic coordination, inventory recovery, delivery state and customer explanation, not commercial readiness.

The catalogue size must also be interpreted correctly. Twenty eight records provide category, price and stock variation for software tests; they do not represent commercial assortment, demand history or recommendation training data. Product ratings and discounts are demonstration values.

Security review is limited to validated request fields, local browser persistence and the absence of payment credentials. A real service would require authenticated ownership checks, encryption policy, rate limiting, dependency scanning, audit events and a documented response to payment or account incidents.

15. Reproducibility and future work

The repository contains the browser application, 28 product catalogue, five public order examples, Java backend, five bounded agents, 14 automated tests, browser verifier, architecture notes, seven experiment figures, screenshots and this report. The frontend and backend suites can be run independently.

A future version should add authenticated accounts, a relational order store, database transactions, expiring reservation records, payment webhooks, courier events, idempotency keys and service monitoring. Full integration testing should then execute the hosted browser against the deployed Java endpoint and verify persisted state across sessions.

Reproduction starts with npm test and the Maven test suite because those commands verify the business rules without a browser. The static site can then be served locally or deployed to Vercel. The browser verifier clears

storage before each run so previous cart, order or inventory values do not hide a regression.

The report figures are generated from declared repository facts and scenario contracts. They are not manually edited dashboard results. Regenerating the figures and PDF after a test count or workflow change keeps the documentation connected to the implementation.

16. Conclusion

BigTech demonstrates a complete electronics order from product discovery through inventory reservation, payment, picking, packing, shipment and delivery. The most important result is that confirmation, payment failure and stock conflict have different, verified side effects rather than different messages over the same state.

The combination of a responsive customer application, deterministic Ezzie support, Java multi agent coordination, automated tests and visible limitations demonstrates software design and failure reasoning without claiming production features that are outside the repository.

The project provides evidence for frontend JavaScript, responsive interface work, Java and Spring Boot structure, REST contract design, state transition reasoning, automated testing, deployment and technical writing. These skills are demonstrated through connected behavior instead of separate sample programs.

The final result is a small but complete lifecycle study. A unit can be purchased, the catalogue stock change remains visible, the delivery path can be followed, a declined payment returns its reservation and a stock conflict never reaches payment. That traceability is the main contribution of BigTech.

The confirmed branch can be followed from start to finish. Inventory records the available quantity, the requested unit is held, payment is approved, the held quantity becomes committed, the delivery estimate is calculated and picking becomes the current milestone. The browser writes the new stock value and the saved order remains available for Ezzie lookup.

The two failure branches provide equally important evidence. A declined payment returns the held unit and prevents fulfilment. A stock conflict rejects the reservation and prevents payment itself. Both attempts remain visible with stopped milestones, which proves safe recovery instead of relying on a temporary alert message.

The separation between customer and technical language is also intentional. Shoppers see stock, payment and delivery terms they recognise. The repository explains Inventory Agent, Payment Agent, Fulfilment Agent, Delivery Agent and Notification Agent because those boundaries are useful for code review, testing and discussion of orchestration.

My final assessment is that the project achieves its declared scope. It is not a commercial store, but it is a reproducible demonstration of how an electronics order should preserve state across success and failure. The implementation, tests, figures, screenshots and limitations all describe the same behavior.